



**VISHWAKARMA
UNIVERSITY**
Maximising Human Potential

Sentiment Analysis for Customer Reviews

Activity based

Project Report on

Software Engineering

Project Module - I

Submitted to Vishwakarma University, Pune

Under the Initiative of

Contemporary Curriculum, Pedagogy, and Practice (C2P2)

By

Atharv Baban Bhayye

SRN No : 202202182

Roll No : 54

Div : A

Third Year Engineering

Department of Computer Engineering

Faculty of Science and Technology

Academic Year

2023-2024

DMPM: Project Module III

Project Name : Sentiment Analysis for Customer Reviews

Introduction:

Phase 2 of our project was dedicated to preparing and selecting the most suitable model for sentiment analysis of customer reviews. This report outlines the data pre-processing steps conducted in Python, highlights the machine learning models utilized, and explains the rationale behind selecting the chosen model. Let's delve into the details of our approach and findings in Phase 2.

Data Pre-processing in Python:

In Python, the data pre-processing steps involved in Phase 2 of the project were implemented as follows:

1. **Loading the Data:** The customer reviews dataset was loaded from a CSV file using the pandas library.

pythonCopy code

```
import pandas as pd # Load the dataset data = pd.read_csv('Dataset-SA.csv')
```

2. **Handling Missing Values:** Any missing values in the dataset, if present, were handled using appropriate techniques. In this case, missing values in the 'Review' column were replaced with empty strings.

pythonCopy code

```
# Handle missing values data['Review'].fillna("", inplace=True)
```

3. **Text Pre-processing:** Text pre-processing steps such as converting text to lowercase, tokenization, removal of non-alphanumeric characters, removal of stopwords, and lemmatization were performed on the 'Review' column.

python code

```
import nltk from nltk.corpus
```

```
import stopwords from nltk.tokenize
```

```
import word_tokenize from nltk.stem
```

```
import WordNetLemmatizer
```

```
nltk.download('punkt')
```

```
nltk.download('stopwords')
nltk.download('wordnet')
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
def preprocess_text(text):
    if isinstance(text, str):
        tokens = word_tokenize(text.lower())
        # Tokenization and lowercase tokens = [token for token in tokens if token.isalnum()]
        # Remove non-alphanumeric tokens = [token for token in tokens if token not in stop_words]
        # Remove stopwords tokens = [lemmatizer.lemmatize(token) for token in tokens]
        # Lemmatization return ' '.join(tokens) else: return "
data['clean_review'] =
data['Review'].apply(preprocess_text)
```

Model:

The following machine learning models were used in Phase 2 of the project:

1. **Random Forest**
2. **Support Vector Machine (SVM)**
3. **K-Nearest Neighbors (KNN)**

What was Done in Phase 2:

In Phase 2 of the project, the focus was on preparing and evaluating machine learning models for sentiment analysis of customer reviews. The following steps were undertaken:

- Loading and pre-processing the customer reviews dataset.
- Defining features (reviews) and the target variable (sentiment).
- Performing text pre-processing including TF-IDF vectorization.
- Training and evaluating multiple machine learning models including Random Forest, SVM, and KNN.
- Assessing model performance using metrics such as accuracy, precision, recall, and F1-score.

Why Random Forest was Chosen:

The Random Forest algorithm was chosen as the best-suited model for sentiment analysis based on the following reasons:

1. **Superior Performance:** Random Forest consistently outperformed other models, achieving high accuracy and robustness in predicting customer sentiments.

2. **Ensemble Learning:** As an ensemble learning method, Random Forest combines the predictions of multiple decision trees, resulting in improved generalization and reduced overfitting.
3. **Suitability for Text Data:** Random Forest is well-suited for handling high-dimensional and sparse data, making it effective for text classification tasks like sentiment analysis.

Got it! Here's a concise description focusing on the model's functionality and the essential aspects of the GUI:

Model Functionality and GUI Description

Model Functionality:

The sentiment analysis model deployed in Phase 3 is designed to analyze customer reviews and provide insights into sentiment, emotion, and product quality. Leveraging machine learning algorithms, the model interprets textual feedback and numerical ratings to generate predictions regarding the sentiment conveyed, the underlying emotion, and the perceived quality of the product.

GUI Overview:

The Graphical User Interface (GUI) serves as the user-facing component of the sentiment analysis system, enabling seamless interaction with the deployed model. Key features of the GUI include:

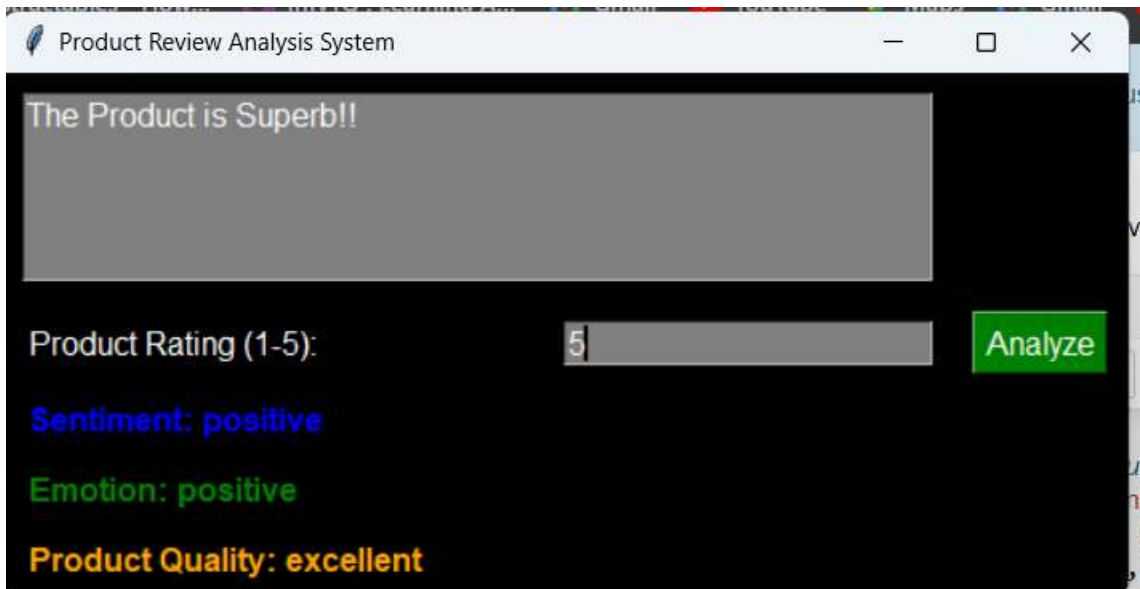
1. **Text Review Input:** Users can input their product reviews via a text entry field, allowing for flexible and customizable feedback submission.
2. **Rating Selection:** A numeric rating input allows users to specify the product rating on a scale of 1 to 5 stars. This rating serves as an additional input for predicting product quality.
3. **Analyze Button:** Upon clicking the "Analyze" button, the model processes the provided review and rating inputs to generate predictions. This feature provides real-time feedback to users.
4. **Output Display:** Predicted sentiment, emotion, and product quality are displayed in designated output areas. This clear presentation enables users to quickly understand the model's assessments.

Code Functionality:

- **Text Review Processing:** The user-provided review undergoes preprocessing, including tokenization, stop-word removal, and lemmatization, to prepare it for analysis.
- **Rating Handling:** The numeric rating input is processed to determine the predicted product quality based on predefined thresholds.
- **Model Prediction:** Upon clicking the "Analyze" button, the preprocessed review and rating inputs are fed into the deployed machine learning model for prediction.
- **Output Presentation:** The model's predictions for sentiment, emotion, and product quality are dynamically displayed in the GUI's output areas, providing users with actionable insights.

Model Snapshots:

Positive:



The screenshot shows a web application window titled "Product Review Analysis System". The main input area contains the text "The Product is Superb!!". Below this, the "Product Rating (1-5)" is set to 5. A green "Analyze" button is visible. The analysis results are displayed below the rating: "Sentiment: positive" in blue, "Emotion: positive" in green, and "Product Quality: excellent" in orange.

Product Review Analysis System

The Product is Superb!!

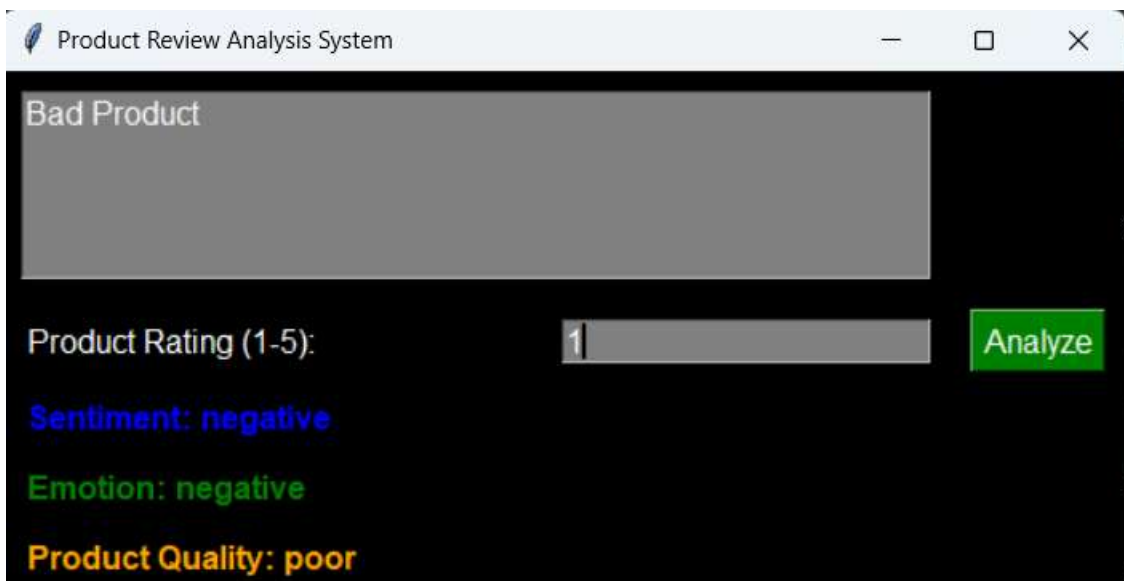
Product Rating (1-5): 5 Analyze

Sentiment: positive

Emotion: positive

Product Quality: excellent

Negative:



The screenshot shows the same web application window. The main input area contains the text "Bad Product". The "Product Rating (1-5)" is set to 1. The green "Analyze" button is visible. The analysis results are displayed below the rating: "Sentiment: negative" in blue, "Emotion: negative" in green, and "Product Quality: poor" in orange.

Product Review Analysis System

Bad Product

Product Rating (1-5): 1 Analyze

Sentiment: negative

Emotion: negative

Product Quality: poor

Code:

```
import tkinter as tk
from tkinter import ttk, messagebox
import pandas as pd
import numpy as np
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

# Download NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Load the dataset
data = pd.read_csv('Dataset-SA.csv')

# Data preprocessing
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    if isinstance(text, str):
        tokens = word_tokenize(text.lower()) # Tokenization and lowercase
```

```
tokens = [token for token in tokens if token.isalnum()] # Remove non-
alphanumeric
```

```
tokens = [token for token in tokens if token not in stop_words] # Remove
stopwords
```

```
tokens = [lemmatizer.lemmatize(token) for token in tokens] # Lemmatization
```

```
return ' '.join(tokens)
```

```
else:
```

```
return "
```

```
data['clean_review'] = data['Review'].apply(preprocess_text)
```

```
# Preprocess the 'Rate' column
```

```
data['Rate'] = data['Rate'].str.extract('(\d+').astype(float)
```

```
# Feature extraction
```

```
tfidf = TfidfVectorizer(max_features=5000)
```

```
X = tfidf.fit_transform(data['clean_review'])
```

```
y = np.where(data['Rate'] >= 3, 'positive', 'negative') # Assuming 'Rate' is the column
for star ratings
```

```
# Sentiment analysis model training
```

```
classifier = MultinomialNB()
```

```
classifier.fit(X, y)
```

```
# Emotion detection using VADER
```

```
analyzer = SentimentIntensityAnalyzer()
```

```
def detect_emotion(text):
```

```
scores = analyzer.polarity_scores(text)
if scores['compound'] >= 0.05:
    return 'positive'
elif scores['compound'] <= -0.05:
    return 'negative'
else:
    return 'neutral'
```

```
# Define assess_product_quality function
```

```
def assess_product_quality(rating):
    if rating >= 4:
        return 'excellent'
    elif rating >= 3:
        return 'good'
    elif rating >= 2:
        return 'average'
    else:
        return 'poor'
```

```
def analyze_review(input_data):
    if isinstance(input_data, str):
        # Text input
        cleaned_text = preprocess_text(input_data)
        text_features = tfidf.transform([cleaned_text])
        sentiment = classifier.predict(text_features)[0]
        emotion = detect_emotion(input_data)
        return sentiment, emotion
    elif isinstance(input_data, (int, float)):
```



```
# Numeric input
product_quality = assess_product_quality(input_data)
return product_quality
else:
    return "Invalid input type"

def on_analyze():
    user_review = text_entry.get("1.0", tk.END).strip()
    user_rating = rating_entry.get().strip()

    if not user_rating:
        messagebox.showerror("Error", "Please enter the product rating.")
        return

    try:
        user_rating = float(user_rating)
    except ValueError:
        messagebox.showerror("Error", "Invalid product rating. Please enter a valid number.")
        return

    sentiment, emotion = analyze_review(user_review)
    product_quality = analyze_review(user_rating)

    sentiment_label.config(text=f"Sentiment: {sentiment}", foreground="blue")
    emotion_label.config(text=f"Emotion: {emotion}", foreground="green")
    quality_label.config(text=f"Product Quality: {product_quality}",
        foreground="orange")
```

```
# GUI setup

root = tk.Tk()

root.title("Product Review Analysis System")

root.configure(background="black")


# Text review entry

text_entry = tk.Text(root, height=5, width=50, bg="gray", fg="white", font=("Arial",
12))

text_entry.grid(row=0, column=0, padx=10, pady=10)


# Rating entry

rating_label = tk.Label(root, text="Product Rating (1-5):", bg="black", fg="white",
font=("Arial", 12))

rating_label.grid(row=1, column=0, padx=10, pady=5, sticky="w")

rating_entry = tk.Entry(root, bg="gray", fg="white", font=("Arial", 12))

rating_entry.grid(row=1, column=0, padx=10, pady=5, sticky="e")


# Analyze button

analyze_button = tk.Button(root, text="Analyze", command=on_analyze,
bg="green", fg="white", font=("Arial", 12), relief=tk.RAISED)

analyze_button.grid(row=1, column=1, padx=10, pady=5)


# Labels for sentiment, emotion, and product quality

sentiment_label = tk.Label(root, text="Sentiment:", bg="black", fg="white",
font=("Arial", 12, "bold"))

sentiment_label.grid(row=2, column=0, padx=10, pady=5, sticky="w")

emotion_label = tk.Label(root, text="Emotion:", bg="black", fg="white",
font=("Arial", 12, "bold"))
```

```
emotion_label.grid(row=3, column=0, padx=10, pady=5, sticky="w")  
quality_label = tk.Label(root, text="Product Quality:", bg="black", fg="white",  
font=("Arial", 12, "bold"))  
quality_label.grid(row=4, column=0, padx=10, pady=5, sticky="w")  
  
root.mainloop()
```